

CEUB

music

O problema



200/s

Alta frequência de escritas em horários de pico, onde cada play exige atualizar a sessão do player em tempo real.

8000

Usuários ativos.

500k

Faixas cadastradas.

14.6Mi

Logs por ano



Por que não relacional?



Joins

Gerar o ranking exige junções com Group By em tempo real; Gargalos.



Contenção de Locks

Incrementar contadores com UPDATE causa bloqueios durante picos simultaneos



Escala Vertical

O crescimento dos logs/ano exige upgrade contínuo de servidor centralizado

Chave-Valor REDIS



Hash + TTL

Sessão do Player:
Armazena o estado ativo
com expiração em 2h.



List

Fila de reprodução:
Estrutura FIFO para as
próximas faixas.



Sorted Set

Ranking Diários:
Atualização atômica via
ZINCRBY e busca do Top
50 em $O(\log N)$.



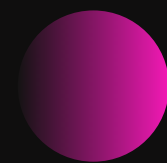
String/INCR

Contadores:
Incremento atômico
global sem contenção
de locks.

Wide-Column Apache Cassandra



Modelagem Query-First:
Tabelas projetadas diretamente para os padrões de acesso da aplicação (Joins)



Partition & Clustering:
Partition Key: usuario_id (isola histórico no mesmo nó);
Clustering Column (reproduzido_em DESC):
Gravação em disco ordenada para leitura instantanea

```
-- Histórico de Reprodução  
  
CREATE TABLE historico_por_usuario (  
    usuario_id int,  
    reproduzido_em timestamp,  
    track_id text,  
    PRIMARY KEY ((usuario_id), reproduzido_em)  
) WITH CLUSTERING ORDER BY (reproduzido_em DESC);
```

CAP, BASE & Distribuição



Opção AP

Prioridade total para Disponibilidade e Tolerância a Partições



Modelo BASE

Soft state via TTL nas sessões e consistência eventual na convergência das réplicas do histórico



Sharding & Replicação

Consistent Hashing no Redis e Murmur3 Partitioner no Cassandra com Fator de Replicação RF=3



Obrigado

Alunos

Arthur Araújo

Daniel Lima

Guilherme Leony

Luan Jywago